Crimson and Thorn Readiness — Snake's Engine vs. the Platformer's Requirements

Date: 23 July 2026 **Scope:** How far the engine currently being rebuilt under Snake is from what `crimson-and-thorn` (Umbra Engine Milestone 2, a 2D narrative platformer) needs — sized against that project's own design docs, not a guess. **Sources:** This branch's source (`src/raylib-facade/`, `src/core/`, `src/engine/`), verified directly rather than assumed; `~/development/projects/fearless-hq/Projects/crimson-and-thorn/` — specifically `umbra-engine-feature-roadmap.md`, `game-design-document.md`, `mechanics-design.md`, and `sprite-and-asset-pipeline.md`; the prior `2026-07-23-engine-completion-status.md` report in this directory, whose Snake punch list this one builds on rather than repeats.

TL;DR

Finishing Snake is the small, nearly-in-reach part of this picture. The platformer's own engine roadmap — written by the project itself, not inferred here — calls for six systems, several of which do not exist **at all** in this codebase today, verified directly against source rather than taken on the roadmap doc's word:

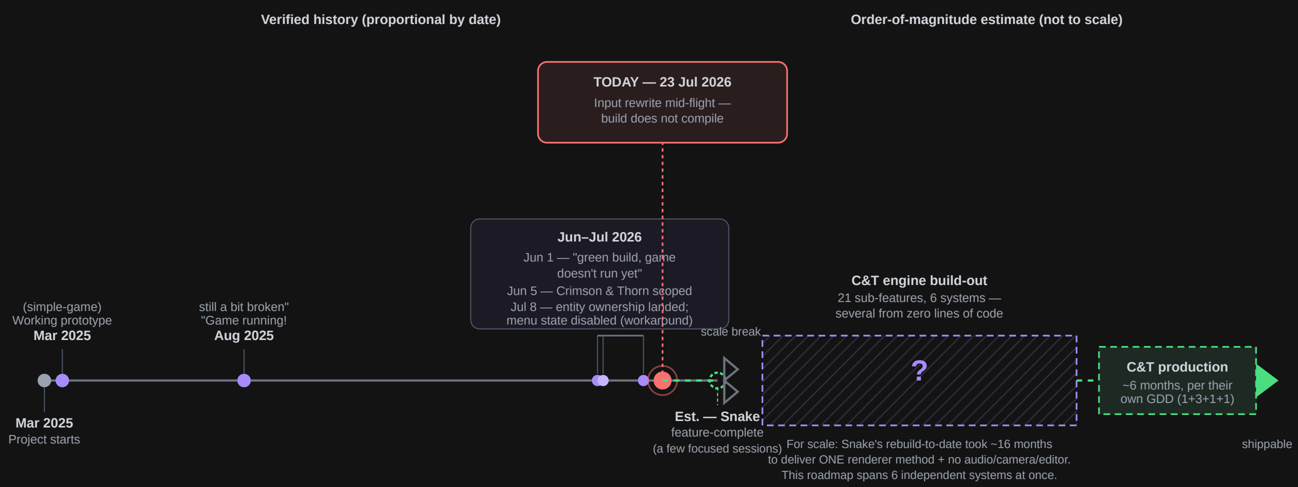
- `IRenderer` / `RaylibRendererFacade` expose exactly four methods — `BeginDrawing` / `EndDrawing` / `ClearBackground` / `DrawRectangle`. No texture loading, no sprites, no camera, no layers, no colour grading, no particles.
- There is no audio code anywhere in `src/` — no interface, no facade, not a stub. Zero.
- There is no `Camera` type anywhere in `src/`.
- `IUserInterface` / `RaylibUserInterfaceFacade` expose exactly one method — `DrawTextCentered`. No font control, no colour, no alignment options.
- There is no persistence/save system (the one `grep` hit for "persist" is an unrelated `SceneLifetime::Persistent` enum value).
- No gamepad support. No JSON library linked in `CMakeLists.txt` yet — needed for both scene serialisation and save data.

Against that, the platformer's own roadmap (`umbra-engine-feature-roadmap.md`) lists **21 distinct sub-features across those 6 systems**, several — a first-party level editor with its own UI, a layered audio mixer, a particle system — that are each roughly the size of an existing engine subsystem Snake spent months building. Snake's own remaining punch list, by comparison, is **13 items**, all mechanical or wiring-level fixes to code that already exists.

The honest scope read: finishing Snake closes out the current chapter. Getting the engine to where Crimson and Thorn's own roadmap says it needs to be is not a continuation of that chapter — it's a comparably-sized *next* one. This report tries to size that next chapter using the platformer's own docs, but says plainly where it can't: several of the roadmap's own line items are still marked "TBD during prototyping" in the source document, so any timeline here is an order-of-magnitude estimate, not a plan.

Where we are in the project lifecycle

The timeline below carries forward from the 23 July status report, extended in both directions: back through the same verified `git log` milestones, and forward past Snake's own estimated finish line into the platformer's requirements. Everything up to and including "Snake feature-complete" is **date-proportional**, same as before. Past that point the axis deliberately **breaks scale** — there's a real difference between "we know the date" and "we're sizing a not-yet-built thing," and drawing both on the same proportional axis would manufacture false precision the platformer's own docs don't claim to have.



Solid = happened. Dashed green = estimate. Hatched purple = order-of-magnitude only, not calendar-scaled.
C&T's own roadmap doc predates Snake's engine having a working build by five weeks (scoped Jun 5, "green build" Jun 1, still broken today).

Reading this left to right: the same story as the 23 July report up through "today," then a genuinely new observation worth sitting with — **Crimson and Thorn's engine roadmap was written on 5 June 2026, four days after the commit literally titled "green build, game doesn't run yet," and seven weeks before today's still-broken build.** The platformer's scope was defined before its own prerequisite had ever run successfully. That's not necessarily a mistake — scoping the destination while still building the road is normal — but it means none of the roadmap's "extend `RaylibRendererFacade`" instructions have yet been tried against a version of that class attached to a working game loop.

What "platformer-ready" requires

Pulled directly from `umbra-engine-feature-roadmap.md`'s own priority order and sub-sections — not reinterpreted, just tabulated:

#	System	Sub-features (roadmap's own breakdown)	Count
1	**Rendering pipeline**	Sprite/texture rendering · sprite animation · layered rendering (bg/mid/fg/UI) · camera system · colour/palette system · particle system	6
2	**Audio system**	Raylib audio facade (`IAudioBackend`) · layered audio mixer (simultaneous tracks, named layers, fades) · audio state system (state → config mapping)	3
3	**Scene editor ("Dawn")**	Architecture (ImGui-backed, its own module) · canvas/viewport · entity placement · layer management · properties panel · JSON serialisation · runtime scene loader	7
4	**Transitions**	Fade to/from black or arbitrary colour, triggered with a completion callback, rendered as a top-level overlay	1
5	**Persistence**	Key-value JSON store (`IPersistenceBackend`) · 3-slot save flow (per `menu-and-save-flow.md`)	2
6	**UI & text rendering**	Font loading/size/colour/alignment · menu & credits UI	2
		Total	**21**

Explicitly **not** counted above, because the project's own docs track them as separate, independently-scoped workstreams rather than engine work:

- **Custom asset packer** (`sprite-and-asset-pipeline.md`) — the prototype phase deliberately uses a third-party tool (TexturePacker) precisely so packer development doesn't block engine/game work; the custom packer is explicitly "tracked as a separate project."
- **Actual game production** (art, levels, audio content, narrative implementation) — the 6-month figure in the timeline diagram above is the platformer's own quoted pre-production → production → testing → release estimate, and it assumes a ready engine as a starting precondition, not something layered on top of engine dev time.

Also surfaced while checking this against the current build (not called out as its own roadmap line item, but a prerequisite for #3 and #5 above): **no JSON library is linked in `CMakeLists.txt` yet** (only `raylib` and `firefly`). Scene serialisation and the save-slot store both need one — this is a small addition, but it's a dependency this repo doesn't have today.

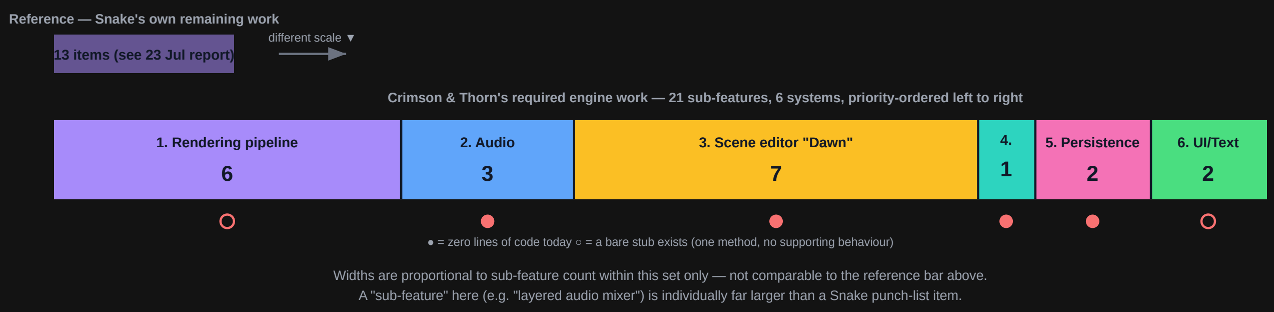
Current engine capability — verified against source, not assumed

Roadmap system	Verified current state
Sprite/texture rendering	**Nothing.** `IRenderer` / `RaylibRendererFacade` (`core/rendering/i-renderer.hpp` , `raylib-facade/renderer/raylib-renderer-facade.hpp`) expose only `BeginDrawing` / `EndDrawing` / `ClearBackground` / `DrawRectangle` . No `LoadTexture` , no sub-rect drawing, no tint.
Sprite animation	**Nothing.** No `AnimationComponent` anywhere in `src/` ; nothing resembling frame-stepping exists.
Layered rendering	**Nothing.** `RenderComponent2DManager` renders whatever's registered with no ordering/layer concept — confirmed by reading the manager directly.
Camera system	**Nothing.** Zero matches for `Camera` anywhere in `src/` .
Colour/palette system	**Partial primitive only.** `ColorRGBA` / `ColorHex` exist (`core/color/`) as static colour values — no interpolation, no named palette states, no per-Chapter tint application.
Particle system	**Nothing.**
Audio (facade, mixer, state system)	**Nothing at all.** Zero matches for audio/sound/music anywhere in `src/` — not a stub, not an empty interface, nothing.
Scene editor ("Dawn")	**Nothing.** No editor module, no ImGui dependency in `CMakeLists.txt` , no serialisation format defined.
Scene loader (JSON → entities)	**Nothing.** `SceneManager` constructs scenes from code today; no data-driven loader.
Transitions	**Nothing.** No fade/overlay concept found.
Persistence	**Nothing.** The only "persist" hit in the codebase is `SceneLifetime::Persistent` , an unrelated scene-lifetime enum value — not a save system.
UI & text rendering	**One method.** `IUserInterface` / `RaylibUserInterfaceFacade` expose exactly `DrawTextCentered(text, position, fontSize)` — no font asset loading, no colour, no alignment control, no menu/credits scaffolding.
Input (relevant groundwork)	**In progress, and directly reusable.** The `Action` / `ActionSet` / `KeyMap` rework this week (see the 23 July status report) is the one piece of current work that isn't a detour — C&T's "run, jump, variable jump height" platforming needs exactly this kind of named, data-driven action system. It's currently mid-rewrite and doesn't compile, but the shape is the right shape.
Gamepad support	**Nothing.** Zero matches for gamepad/joystick anywhere in `src/` .

Twelve of thirteen rows above read "nothing" or "one method." This isn't a criticism of pace — Snake's engine wasn't built to have these things yet, it was built to run a snake around a grid — it's the actual distance being measured here.

Scope comparison

Two different things, deliberately **not drawn on the same proportional scale** — a Snake compile-fix and "build a first-party level editor" are not comparable units of effort, and a chart implying otherwise would be more misleading than useful.



What this means, plainly

- **Snake finishing is real and close.** The 13-item punch list from the 23 July report stands — mechanical compile fixes, then reconnecting logic that already exists. Nothing here changes that estimate.
- **The platformer is not "Snake plus a few features."** Two of its six required systems (rendering, scene editor) are individually larger than what exists in this engine today. Audio doesn't exist as a concept in this codebase at all. This is a second build-out, not a continuation of the current one.
- **The roadmap itself doesn't claim precision, and neither should this report.** Its own "Open Questions" table flags atlas strategy, particle complexity, and audio middleware as unresolved; `mechanics-design.md` and `sprite-and-asset-pipeline.md` each carry their own "to be resolved via prototyping" lists. Any date attached to "engine build-out" in the timeline above is a magnitude signal, not a commitment — which is exactly why it's drawn hatched and question-marked rather than as a dashed dot like Snake's estimate.
- **One piece of current work is genuinely dual-purpose.** The input-action rework happening in Snake right now (`Action` / `ActionSet` / `KeyMap`) is architecturally what C&T's platforming input needs too — named, data-driven actions rather than a hardcoded enum. Finishing it isn't a detour from the platformer goal, even though it's being built for Snake first.
- **The order of magnitude, stated once, plainly:** Snake's engine took roughly sixteen months to reach "green build, game doesn't run yet" while building one renderer method and no audio, camera, or editor. Crimson and Thorn's roadmap asks for six independent systems, several from zero, one of which (Dawn) is itself a small application in its own right. Treating the next phase as comparable in scale to the whole Snake rebuild to date — not a quick follow-on to it — is the honest way to size this.

Not rendered to PDF. Run `/report-pdf` against this file when a shareable copy is wanted.